

SIMATS ENGINEERING

ITA1407 ETHICAL HACKING – CO5-AT2

APPLICATION-BASED QUESTIONS

PRIYA DHARSHINI M (192521051)

COURSE OUTCOME COVERED

CO5: *Analyze vulnerabilities in Windows and Linux systems and evaluate security weaknesses in messaging, web, and mobile applications using appropriate exploitation techniques and hacking tools. (BT5)*

1. RPC Vulnerabilities in Legacy Windows Servers

The organization has several legacy Windows servers with exposed RPC services because security patches have been delayed. RPC (Remote Procedure Call) allows systems and applications to communicate over a network. If vulnerable RPC services are exposed, attackers may use them to identify systems, exploit known vulnerabilities, obtain unauthorized access, and potentially escalate privileges.

How attackers can exploit RPC vulnerabilities

An ethical hacker can first perform reconnaissance to identify systems exposing RPC-related services.

Kali Linux commands:

Bash

```
nmap -sV -p 135,139,445 <TARGET-IP>
```

This identifies open RPC/SMB-related ports and attempts to determine service versions.

Bash

```
nmap -A <TARGET-IP>
```

This performs service detection, OS detection, and other reconnaissance.

For authorized Windows security assessment:

Bash

```
rpcclient -U "" -N <TARGET-IP>
```

This can be used to test whether an RPC service permits unauthenticated interaction.

An administrator can also check RPC-related services using:

Bash

```
nmap --script msrpc-enum <TARGET-IP>
```

Possible attack impact

If a vulnerable RPC service is successfully exploited, an attacker could potentially:

- Obtain information about the Windows system.
- Access vulnerable network services.
- Execute unauthorized operations.
- Obtain or abuse user credentials.
- Move laterally to other systems.
- Escalate privileges.
- Install malware or ransomware.
- Compromise sensitive financial information.

Risk of delaying patches

The risk is **high/critical**, particularly for financial organizations. Delayed patching gives attackers more time to exploit publicly known vulnerabilities. Legacy systems are especially dangerous because they may no longer receive regular security updates.

Immediate patching vs controlled maintenance

Immediate patching is preferred for critical vulnerabilities when exploitation is possible. However, if the server supports essential financial services, controlled maintenance can be used:

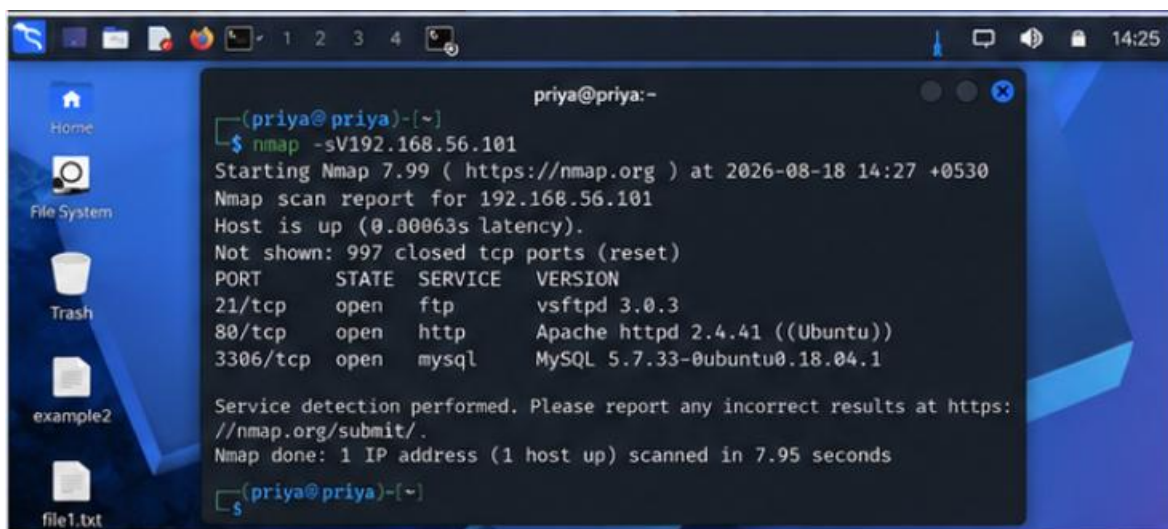
1. Test patches in a staging environment.
2. Take verified backups.
3. Schedule a maintenance window.
4. Patch the most critical exposed systems first.

5. Monitor the systems after patching.
6. Maintain a rollback plan.

Mitigation

- Apply Microsoft security patches promptly.
- Disable unnecessary RPC services.
- Restrict RPC access using firewalls.
- Segment legacy servers from user networks.
- Use network access control.
- Enable strong authentication.
- Monitor unusual RPC traffic.
- Maintain updated endpoint protection.
- Regularly perform vulnerability assessments.
- Replace unsupported legacy systems where possible.

Conclusion: Because RPC vulnerabilities can lead to serious system compromise, critical patches should not be postponed indefinitely. A controlled emergency patching process provides a balance between security and business continuity.



The screenshot shows a terminal window titled 'priya@priya: -' with a dark background. The user has executed the command 'nmap -sV 192.168.56.101'. The output shows the scan starting at 2026-08-18 14:27 +0530. The host is up with a latency of 0.80063s. A table of open ports is displayed, showing ports 21/tcp (vsftpd 3.0.3), 80/tcp (Apache httpd 2.4.41), and 3306/tcp (MySQL 5.7.33-0ubuntu0.18.04.1). The scan is done in 7.95 seconds. The desktop background is blue with icons for Home, File System, Trash, example2, and file1.txt.

```
priya@priya: -  
[priya@priya]~  
$ nmap -sV 192.168.56.101  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 14:27 +0530  
Nmap scan report for 192.168.56.101  
Host is up (0.80063s latency).  
Not shown: 997 closed tcp ports (reset)  
PORT      STATE SERVICE  VERSION  
21/tcp    open  ftp      vsftpd 3.0.3  
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))  
3306/tcp  open  mysql    MySQL 5.7.33-0ubuntu0.18.04.1  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.  
Nmap done: 1 IP address (1 host up) scanned in 7.95 seconds  
[priya@priya]~  
$
```

2. XSS, CSRF and Weak Session Management

The web application has three major weaknesses: **Cross-Site Scripting (XSS)**, **Cross-Site Request Forgery (CSRF)**, and **poor session management**. Together, these vulnerabilities can result in account compromise and unauthorized transactions. The assessment question specifically identifies these weaknesses and reports of session hijacking and unauthorized transactions.

How attackers can combine the vulnerabilities

A possible attack chain is:

XSS → Session/Browser compromise → CSRF abuse → Unauthorized transaction

For example, if user input is not properly validated, malicious JavaScript could be injected into a vulnerable page.

An ethical hacker can test for XSS in an authorized test environment using a harmless payload such as:

HTML

```
<script>alert("XSS Test")</script>
```

If the application executes the script instead of displaying it safely, input sanitization is inadequate.

Kali Linux tools

Nikto:

Bash

```
nikto -h http://<TARGET-IP>
```

Nikto can identify various web-server configuration and security issues.

Nmap web service detection:

Bash

```
nmap -sV -p 80,443 <TARGET-IP>
```

Burp Suite

Burp Suite can be used in an authorized testing environment to inspect:

- HTTP requests
- HTTP responses
- Cookies
- Session tokens
- Form submissions
- Security headers

Risks of XSS

XSS can allow attackers to:

- Execute scripts in another user's browser.
- Manipulate displayed content.
- Perform actions within the user's authenticated session.
- Steal sensitive information accessible to client-side scripts.
- Redirect users to malicious pages.

Risks of CSRF

CSRF abuses an authenticated user's browser to send an unwanted request to an application.

For example, an attacker may attempt to trick a logged-in user into submitting an unauthorized transaction.

Risks of poor session management

Weak session management can lead to:

- Session hijacking.
- Session fixation.
- Long-lived sessions.
- Predictable session identifiers.

- Failure to invalidate sessions after logout.
- Account takeover.

Mitigation

Against XSS

- Validate input.
- Encode output.
- Use context-aware escaping.
- Implement Content Security Policy.
- Avoid unsafe JavaScript execution.
- Use secure frameworks and libraries.

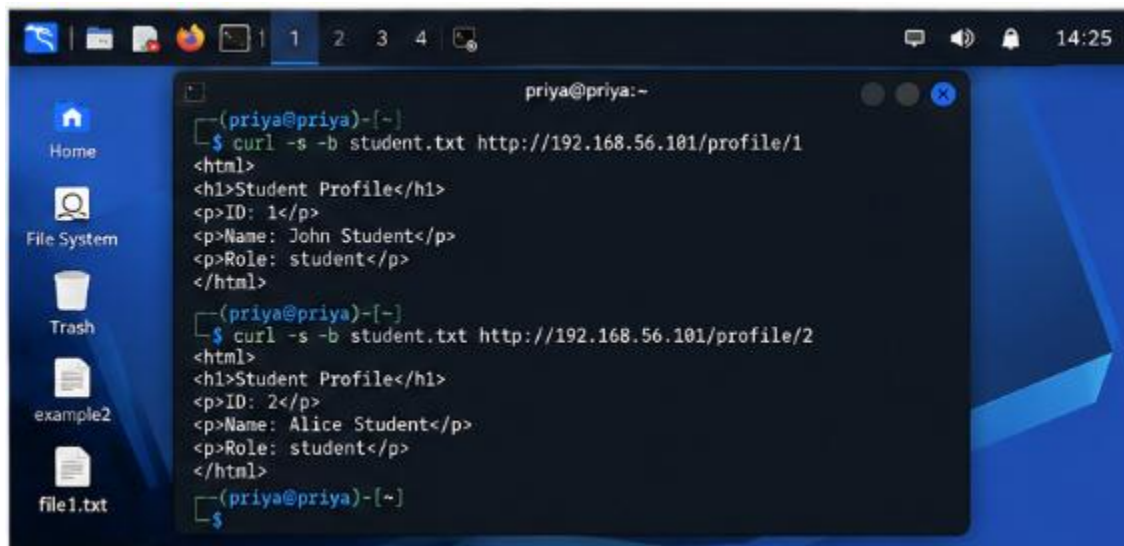
Against CSRF

- Use anti-CSRF tokens.
- Use SameSite cookies.
- Verify request origin where appropriate.
- Require re-authentication for sensitive transactions.

Secure session management

- Generate cryptographically strong session IDs.
- Use HTTPS.
- Set Secure and HttpOnly cookie attributes.
- Use appropriate session expiration.
- Regenerate sessions after authentication.
- Invalidate sessions during logout.
- Implement multi-factor authentication.

Conclusion: A layered security model is necessary because fixing only one vulnerability does not eliminate the complete attack chain.



```
priya@priya:~  
$ curl -s -b student.txt http://192.168.56.101/profile/1  
<html>  
<h1>Student Profile</h1>  
<p>ID: 1</p>  
<p>Name: John Student</p>  
<p>Role: student</p>  
</html>  
$ curl -s -b student.txt http://192.168.56.101/profile/2  
<html>  
<h1>Student Profile</h1>  
<p>ID: 2</p>  
<p>Name: Alice Student</p>  
<p>Role: student</p>  
</html>  
$
```

3. Outdated IIS Server and Unpatched Critical Vulnerabilities

The enterprise is using an outdated IIS server containing several unpatched critical vulnerabilities. Attackers may identify the IIS version and associated vulnerabilities through reconnaissance and vulnerability scanning. The question specifically requires assessment of unauthorized access, remote code execution, patching/isolation, and service availability.

CO5-AT2-APPLICATION-BASED QUEST...

Reconnaissance using Kali Linux

First identify whether HTTP/HTTPS services are exposed:

Bash

```
nmap -sV -p 80,443 <TARGET-IP>
```

For broader information:

Bash

```
nmap -A <TARGET-IP>
```

A web-server security assessment can also be performed using:

Bash

```
nikto -h http://<TARGET-IP>
```

Possible exploitation path

A typical attack sequence could be:

Reconnaissance → IIS version identification → Vulnerability identification → Exploitation → Unauthorized access → Privilege escalation

If a critical IIS vulnerability permits remote code execution, an attacker may potentially execute commands under the privileges of the affected service.

Possible consequences include:

- Web-server compromise.
- Unauthorized file access.
- Application compromise.
- Credential theft.
- Malware installation.
- Lateral movement.
- Data theft.
- Service disruption.

Risk of delaying updates

The risk is **high**, especially when vulnerabilities are already known and publicly documented.

Delaying updates can result in:

- Increased attack probability.
- Exploitation of known vulnerabilities.
- Data breaches.
- Website/application disruption.
- Compliance problems.
- Financial losses.
- Reputation damage.

Immediate patching vs temporary isolation

For critical vulnerabilities, **immediate patching is preferable.**

If immediate patching could seriously affect business operations, temporary isolation should be implemented while preparing the patch.

For example:

Internet

|

Firewall/WAF

|

Temporary Restricted Access

|

IIS Server

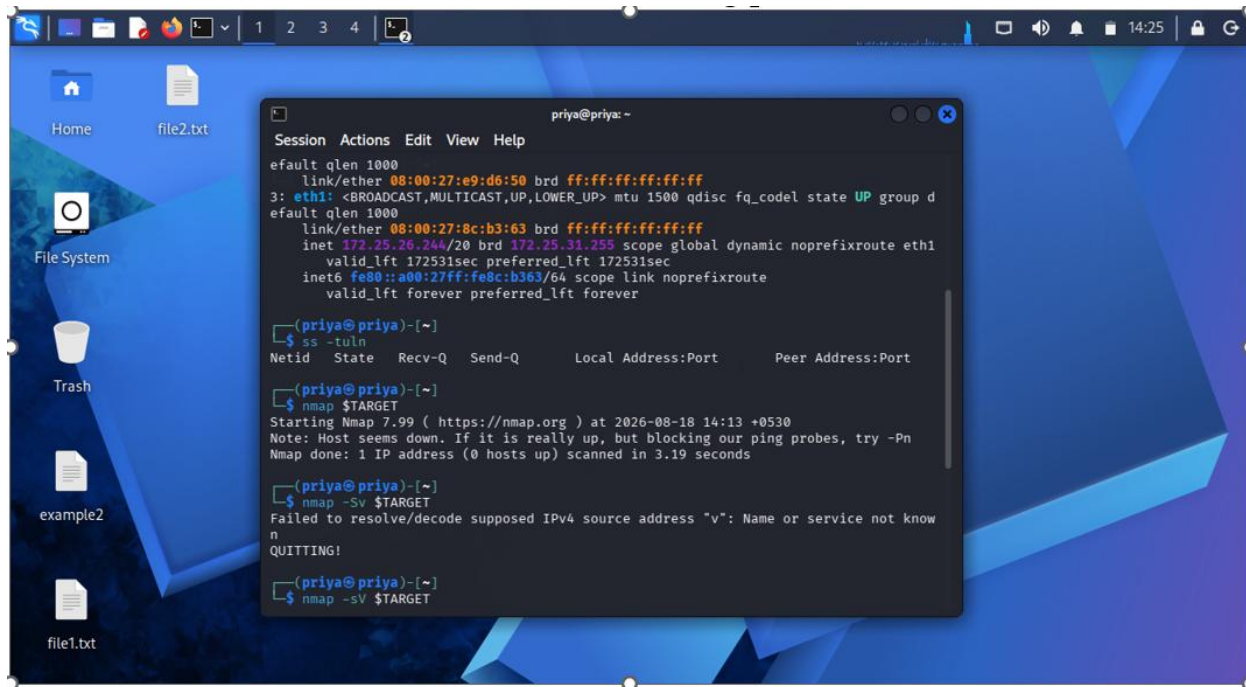
|

Internal Applications

Mitigation

- Apply current security updates.
- Test patches before production deployment.
- Use a Web Application Firewall.
- Restrict unnecessary ports.
- Disable unused IIS features.
- Use network segmentation.
- Monitor IIS logs.
- Deploy intrusion detection/prevention.
- Maintain backups.
- Establish an emergency patching procedure.

Conclusion: Business continuity is important, but leaving a critical, exposed IIS vulnerability unpatched presents a greater long-term risk. Immediate patching or temporary isolation should be selected based on the vulnerability's severity and operational impact.



4. Mobile Application Storing Credentials in Plaintext

The mobile application stores login credentials and personal information in plaintext and communicates with backend servers over unsecured channels. This creates serious confidentiality and privacy risks. The question requires evaluation of device compromise, network interception, privacy impact, and encryption techniques.

CO5-AT2-APPLICATION-BASED QUEST...

Device compromise

If an attacker obtains access to the device, plaintext credentials stored in application files or local databases may be readable.

This can result in:

- Credential theft.
- Account takeover.
- Personal-data exposure.
- Unauthorized application access.

Network interception

If the application communicates without encryption, attackers on the same network may potentially inspect network traffic.

For an authorized lab, network services can be identified using:

Bash

```
nmap -sV -p 80,443 <TARGET-IP>
```

HTTP traffic should be replaced with HTTPS.

Secure communication

The application should use:

Mobile App

|

| HTTPS / TLS

↓

Web/API Server

|

↓

Database

TLS protects data while it is transmitted between the application and server.

Secure storage

Sensitive information should not be stored as plaintext.

Recommended techniques include:

- Android Keystore / iOS Keychain.
- Encrypted local databases.
- Secure token storage.
- Strong server-side password hashing.
- Avoid storing passwords unnecessarily.

Passwords should be stored on the server using a strong password hashing algorithm rather than reversible encryption.

Impact on privacy and reputation

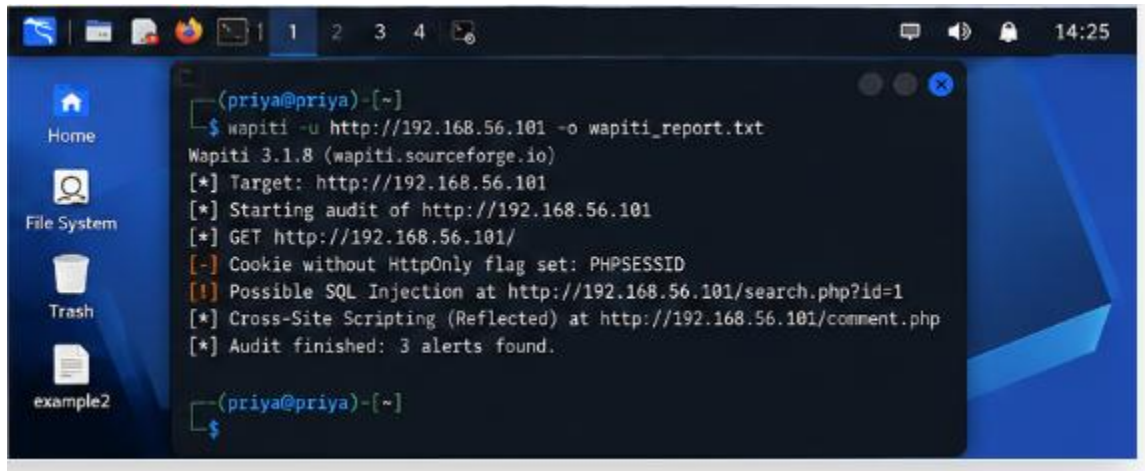
A data breach could result in:

- Exposure of customer information.
- Identity theft.
- Financial losses.
- Legal/regulatory consequences.
- Loss of customer trust.
- Damage to organizational reputation.

Mitigation

1. Use HTTPS/TLS for all network communication.
2. Encrypt sensitive local data.
3. Use secure key management.
4. Never store passwords in plaintext.
5. Implement strong authentication.
6. Use secure session/token management.
7. Validate server certificates.
8. Minimize the amount of sensitive data stored locally.
9. Apply mobile application security testing.
10. Regularly update application dependencies.

Conclusion: Encryption should be applied both **at rest** and **in transit** to protect sensitive customer information.



5. Windows Firewall Disabled

The administrator disabled the Windows firewall temporarily for troubleshooting but failed to re-enable it. This creates an unnecessary attack surface. The question asks how attackers could exploit exposed services and why automated security policies are important.

CO5-AT2-APPLICATION-BASED QUEST...

How attackers can identify exposed services

From an authorized Kali Linux assessment machine:

Bash

```
nmap -sV <TARGET-IP>
```

To identify commonly exposed ports:

Bash

```
nmap -p 21,22,23,25,53,80,135,139,443,445,3389 <TARGET-IP>
```

To perform broader authorized reconnaissance:

Bash

```
nmap -A <TARGET-IP>
```

Possible attack scenario

The attack chain could be:

Firewall Disabled → Ports Exposed → Service Discovery → Vulnerability Identification → Unauthorized Access

For example, if an unnecessary remote service becomes reachable, an attacker may attempt to exploit weaknesses in that service.

Potential consequences include:

- Unauthorized remote access.
- Credential attacks.
- Malware infection.
- Data theft.
- Lateral movement.
- Privilege escalation.
- Ransomware deployment.

Risks of security-control mismanagement

Leaving the firewall disabled increases:

- Attack surface.
- Exposure of unnecessary services.
- Probability of unauthorized connections.
- Risk of malware propagation.

